# Recipe App — Source Code Reference

A file-by-file reference of every source file in the project, with a brief description of each file's role.

---

## Root

### `package.json`

Declares project metadata, dependencies, and npm scripts. Key scripts: - `dev` — starts the Vite development server - `server` — starts JSON Server watching `db.json` on port 3002 - `build` — compiles the app for production - `lint` — runs ESLint across all source files

```
{
  "name": "recipe-app",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "server": "json-server --watch db.json --port 3002",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.4",
    "react-dom": "^19.2.4",
    "react-router-dom": "^7.9.4"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.4",
    "@types/react": "^19.2.14",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.1",
    "eslint": "^9.39.4",
    "eslint-plugin-react-hooks": "^7.0.1",
    "eslint-plugin-react-refresh": "^0.5.2",
    "globals": "^17.4.0",
    "json-server": "^1.0.0-beta.3",
    "vite": "^8.0.1"
  }
}
```

---

`db.json`

The JSON Server database file. JSON Server reads and writes this file directly at runtime. The `recipes` array contains one seeded sample recipe on first run.

Each recipe object shape: | Field | Type | Description | |—|—|—| | `id` | string | Unique identifier (assigned by JSON Server on POST) | | `name` | string | Recipe title | | `category` | string | Meal type (Breakfast, Lunch, Dinner, Snack, Dessert, Drink) | | `description` | string | One-line summary | | `ingredients` | string[] | List of ingredient strings | | `instructions` | string[] | Ordered list of steps | | `prepTime` | number | Preparation time in minutes | | `cookTime` | number | Cook time in minutes | | `servings` | number | Number of servings |

```
{
  "recipes": [
    {
      "id": "seed-1",
      "name": "Lemon Herb Chickpea Bowl",
      "category": "Lunch",
      "description": "Bright, protein-rich bowl with chickpeas, quinoa, and c
itrus herbs.",
      "ingredients": [
        "1 cup cooked quinoa",
        "1 can chickpeas, rinsed",
        "1 cup cherry tomatoes, halved",
        "1 cucumber, diced",
        "2 tbsp olive oil",
        "1 lemon, juiced",
        "2 tbsp chopped parsley",
        "Salt and black pepper"
      ],
      "instructions": [
        "In a large bowl, whisk olive oil, lemon juice, salt, and pepper.",
        "Add quinoa, chickpeas, tomatoes, and cucumber; toss well.",
        "Fold in chopped parsley and adjust seasoning.",
        "Serve chilled or at room temperature."
      ],
      "prepTime": 15,
      "cookTime": 0,
      "servings": 2
    }
  ]
}
```

---

`src/`

`main.jsx`

The React entry point. Mounts the app into `index.html`'s `#root` element wrapped in three top-level providers: - `StrictMode` — highlights potential React issues during development - `BrowserRouter` — enables client-side routing with the History API - `RecipesProvider` — makes shared recipe state available to every component in the tree

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import "./index.css";
import App from "./App.jsx";
import { RecipesProvider } from "./hooks/useRecipes";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <RecipesProvider>
        <App />
      </RecipesProvider>
    </BrowserRouter>
  </StrictMode>,
);
```

---

`App.jsx`

Defines the route map using React Router's `<Routes>` and `<Route>` components. All routes are nested under the shared `Layout` route so the header and navigation persist across page changes. Any unknown path redirects to Home.

```
import { Navigate, Route, Routes } from "react-router-dom";
import Layout from "./components/Layout";
import AddRecipe from "./pages/AddRecipe";
import BrowseRecipes from "./pages/BrowseRecipes";
import Home from "./pages/Home";
import RecipeDetail from "./pages/RecipeDetail";
import "./App.css";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Layout />}>
        <Route index element={<Home />} />
        <Route path="add" element={<AddRecipe />} />
        <Route path="recipes" element={<BrowseRecipes />} />
        <Route path="recipes/:id" element={<RecipeDetail />} />
```

```
        <Route path="*" element={<Navigate to="/" replace />} />
      </Route>
    </Routes>
  );
}

export default App;
```

---

`index.css`

Global stylesheet imported in `main.jsx`. Defines: - CSS custom properties (design tokens) for colours, panel backgrounds, borders, and accents - Body background using `radial-gradient` blobs and a linear gradient for a soft, warm feel - Base typography — `Fraunces` (serif) for headings, `Manrope` (sans-serif) for body text

```
@import url("https://fonts.googleapis.com/css2?family=Fraunces:opsz,wght@9..144,500;9..144,700&family=Manrope:wght@400;500;700&display=swap");

:root {
  --bg-top: #fff7ea;
  --bg-bottom: #eaf8ff;
  --text-main: #222435;
  --text-soft: #5c6275;
  --panel-bg: rgba(255, 255, 255, 0.86);
  --panel-border: #d9e1e8;
  --accent-1: #d1642d;
  --accent-2: #0d8c7c;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  min-height: 100vh;
  color: var(--text-main);
  font-family: "Manrope", "Segoe UI", sans-serif;
  background:
    radial-gradient(circle at 9% 15%, rgba(255, 202, 133, 0.45), transparent 24%),
    radial-gradient(circle at 92% 80%, rgba(126, 227, 208, 0.4), transparent 22%),
    linear-gradient(140deg, var(--bg-top), var(--bg-bottom));
}

h1, h2, h3, h4 {
  font-family: "Fraunces", Georgia, serif;
```

```
  line-height: 1.15;
}
```

---

### App.css

All component and layout styles. Notable classes: - `.app-shell` / `.app-header` — max-width container and branded glass-panel header - `.panel` / `.hero-panel` — reusable card surface; hero variant uses a warm gradient - `.button` / `.button-ghost` / `.button-danger` — three button styles (primary, outlined, destructive) - `.recipe-grid` — responsive `auto-fit` grid for browse cards - `.recipe-form` / `.time-grid` — form layout with a 3-column row for time/servings inputs - `.filter-bar` — 2-column grid for the search input and category dropdown - `.details-grid` — 2-column ingredient/instruction layout on detail page - `.message.error` / `.message.success` — inline status banners - Media query at `max-width: 760px` collapses multi-column grids to a single column

---

## src/data/

### recipeSeed.js

Exports two constants shared by `RecipeForm` and `BrowseRecipes`: - `CATEGORIES` — the fixed list of meal categories shown in dropdowns - `EMPTY_RECIPE_FORM` — the blank form state object used to reset the add form after submission

```
export const CATEGORIES = ["Breakfast", "Lunch", "Dinner", "Snack", "Dessert",
 "Drink"];

export const EMPTY_RECIPE_FORM = {
  name: "",
  category: "Dinner",
  description: "",
  ingredients: "",
  instructions: "",
  prepTime: "",
  cookTime: "",
  servings: "",
};
```

---

## src/services/

### recipesApi.js

Fetch-based API client targeting JSON Server at `http://localhost:3002`. All functions are `async` and throw descriptive errors on non-OK responses so consuming components can surface them to the user.

| Function | Method | Description |
|---|---|---|
| `listRecipes()` | GET | Fetch all recipes |
| `getRecipeById(id)` | GET | Fetch one recipe by id; returns `null` on 404 |
| `createRecipe(data)` | POST | Add a new recipe; returns the saved object with its assigned id |
| `updateRecipe(id, data)` | PUT | Replace a recipe by id; returns the updated object |
| `deleteRecipe(id)` | DELETE | Remove a recipe by id; returns `true` on success |

```
const API_BASE_URL = "http://localhost:3002";
const RECIPES_ENDPOINT = `${API_BASE_URL}/recipes`;

async function parseResponse(response, action) {
  if (!response.ok) {
    throw new Error(`Unable to ${action}. Please check that JSON Server is ru
nning.`);
  }
  return response.json();
}

export async function listRecipes() {
  const response = await fetch(RECIPES_ENDPOINT);
  return parseResponse(response, "load recipes");
}

export async function getRecipeById(recipeId) {
  const response = await fetch(`${RECIPES_ENDPOINT}/${recipeId}`);
  if (response.status === 404) return null;
  return parseResponse(response, "load this recipe");
}

export async function createRecipe(recipeData) {
  const response = await fetch(RECIPES_ENDPOINT, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(recipeData),
  });
  return parseResponse(response, "save your recipe");
}

export async function updateRecipe(recipeId, recipeData) {
  const response = await fetch(`${RECIPES_ENDPOINT}/${recipeId}`, {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
```

```
    body: JSON.stringify(recipeData),
  });
  return parseResponse(response, "update this recipe");
}

export async function deleteRecipe(recipeId) {
  const response = await fetch(`${RECIPES_ENDPOINT}/${recipeId}`, { method: "
DELETE" });
  if (!response.ok) {
    throw new Error("Unable to delete this recipe. Please check that JSON Ser
ver is running.");
  }
  return true;
}
```

---

`src/hooks/`

`useRecipes.js`

Implements the `RecipesProvider` context component and the `useRecipes` consumer hook.

**`normalizeRecipe(formData)`** (internal helper) Converts the string-based form state into the recipe object shape expected by the API: - Trims whitespace from name and description - Splits ingredients and instructions textareas on commas or newlines into arrays - Converts time and servings strings to numbers

**`RecipesProvider`** Context provider that should wrap the whole app (mounted in `main.jsx`). On mount it fetches all recipes and stores them in state. Exposes: | Value | Type | Description | |—|—|—| | `recipes` | array | All recipes currently loaded | | `isLoading` | boolean | True while the initial fetch is in progress | | `error` | string | Error message if the fetch failed | | `addRecipe(formData)` | async fn | Normalises form data, POSTs to API, prepends result to state | | `editRecipe(id, formData)` | async fn | Normalises form data, PUTs to API, replaces matching item in state | | `removeRecipe(id)` | async fn | DELETEs from API, filters item out of state | | `refreshRecipes()` | async fn | Re-fetches all recipes from the API |

**`useRecipes()`** Consuming hook. Throws a descriptive error if called outside `RecipesProvider`.

```
import {
  createContext, createElement, useCallback,
  useContext, useEffect, useMemo, useState,
} from "react";
import { createRecipe, deleteRecipe, listRecipes, updateRecipe } from "../ser
vices/recipesApi";

const RecipesContext = createContext(null);

function normalizeRecipe(formData) {
```

```
  const splitLines = (value) =>
    value.split(/\r?\n|,/).map((item) => item.trim()).filter(Boolean);
  return {
    name: formData.name.trim(),
    category: formData.category,
    description: formData.description.trim(),
    ingredients: splitLines(formData.ingredients),
    instructions: splitLines(formData.instructions),
    prepTime: Number(formData.prepTime) || 0,
    cookTime: Number(formData.cookTime) || 0,
    servings: Number(formData.servings) || 1,
  };
}

export function RecipesProvider({ children }) {
  const [recipes, setRecipes] = useState([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState("");

  const loadRecipes = useCallback(async () => {
    setIsLoading(true); setError("");
    try {
      const data = await listRecipes();
      setRecipes(data);
    } catch (e) { setError(e.message); }
    finally { setIsLoading(false); }
  }, []);

  useEffect(() => { loadRecipes(); }, [loadRecipes]);

  const addRecipe = useCallback(async (formData) => {
    const saved = await createRecipe(normalizeRecipe(formData));
    setRecipes((c) => [saved, ...c]);
    return saved;
  }, []);

  const editRecipe = useCallback(async (id, formData) => {
    const updated = await updateRecipe(id, normalizeRecipe(formData));
    setRecipes((c) => c.map((r) => String(r.id) === String(id) ? updated :
r));
    return updated;
  }, []);

  const removeRecipe = useCallback(async (id) => {
    await deleteRecipe(id);
    setRecipes((c) => c.filter((r) => String(r.id) !== String(id)));
  }, []);

  const value = useMemo(
```

```
    () => ({ recipes, isLoading, error, addRecipe, editRecipe, removeRecipe,
refreshRecipes: loadRecipes }),
    [recipes, isLoading, error, addRecipe, editRecipe, removeRecipe, loadReci
pes],
  );

  return createElement(RecipesContext.Provider, { value }, children);
}

export function useRecipes() {
  const ctx = useContext(RecipesContext);
  if (!ctx) throw new Error("useRecipes must be used within RecipesProvider");
  return ctx;
}
```

---

`src/components/`

`Layout.jsx`

Persistent shell rendered by the parent `<Route>` in `App.jsx`. Renders the branded header and uses `<Outlet />` as the slot where child page components are injected. `NavLink` applies the `nav-link-active` class to the link matching the current route.

```
import { NavLink, Outlet } from "react-router-dom";

const NAV_ITEMS = [
  { to: "/", label: "Home" },
  { to: "/add", label: "Add Recipe" },
  { to: "/recipes", label: "Browse Recipes" },
];

function Layout() {
  return (
    <div className="app-shell">
      <header className="app-header">
        <p className="brand-kicker">Kitchen Journal</p>
        <h1>Recipe Studio</h1>
        <p className="brand-copy">Create, discover, and save your go-to dishe
s.</p>
        <nav className="app-nav" aria-label="Main navigation">
          {NAV_ITEMS.map((item) => (
            <NavLink key={item.to} to={item.to}
              className={({ isActive }) => isActive ? "nav-link nav-link-acti
ve" : "nav-link"}>
              {item.label}
            </NavLink>
          ))}
        </nav>
```

```
      </header>
      <main className="app-main">
        <Outlet />
      </main>
    </div>
  );
}

export default Layout;
```

---

`RecipeCard.jsx`

Presentational component used in the browse grid. Receives a `recipe` prop and renders category, name, description, the first three ingredients as a preview, total time, and a "View Recipe" link to the detail page.

```
import { Link } from "react-router-dom";

function RecipeCard({ recipe }) {
  const ingredientPreview = recipe.ingredients.slice(0, 3).join(", ");
  return (
    <article className="recipe-card">
      <p className="recipe-category">{recipe.category}</p>
      <h3>{recipe.name}</h3>
      <p>{recipe.description}</p>
      <p className="recipe-preview">{ingredientPreview}</p>
      <div className="recipe-meta">
        <span>{recipe.prepTime + recipe.cookTime} min total</span>
        <span>{recipe.servings} servings</span>
      </div>
      <Link className="button button-ghost" to={`/recipes/${recipe.id}`}>View
 Recipe</Link>
    </article>
  );
}

export default RecipeCard;
```

---

`RecipeForm.jsx`

Controlled form component shared by both the Add Recipe page and the inline edit mode in Recipe Detail.

**Props** | Prop | Type | Default | Description | |—|—|—|—| | `onSubmit` | async fn | required | Called with the current form state on valid submit | | `isSubmitting` | boolean | required | Disables the submit button and shows "Saving…" | | `initialData` | object | null | `null` | If

provided, pre-populates the form (used for edit) | | `submitLabel` | string | `"Save Recipe"` | Button label | | `successMessage` | string | `"Recipe saved successfully."` | Message shown after a successful submit | | `resetOnSuccess` | boolean | `true` | Whether to clear the form after success (false for edit) |

The internal `toFormData` helper converts a stored recipe (with array fields) back into textarea-friendly newline-separated strings when `initialData` is provided.

```
import { useState } from "react";
import { CATEGORIES, EMPTY_RECIPE_FORM } from "../data/recipeSeed";

function toFormData(initialData) {
  if (!initialData) return EMPTY_RECIPE_FORM;
  return {
    name: initialData.name ?? "",
    category: initialData.category ?? "Dinner",
    description: initialData.description ?? "",
    ingredients: Array.isArray(initialData.ingredients)
      ? initialData.ingredients.join("\n") : (initialData.ingredients ?? ""),
    instructions: Array.isArray(initialData.instructions)
      ? initialData.instructions.join("\n") : (initialData.instructions ?? "
"),
    prepTime: initialData.prepTime ?? "",
    cookTime: initialData.cookTime ?? "",
    servings: initialData.servings ?? "",
  };
}

function RecipeForm({ onSubmit, isSubmitting, initialData = null,
  submitLabel = "Save Recipe", successMessage = "Recipe saved successfully.",
 resetOnSuccess = true }) {

  const [formData, setFormData] = useState(toFormData(initialData));
  const [message, setMessage] = useState("");
  const [formError, setFormError] = useState("");

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData((c) => ({ ...c, [name]: value }));
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setFormError(""); setMessage("");
    if (!formData.name.trim() || !formData.ingredients.trim() || !formData.in
structions.trim()) {
      setFormError("Please add a name, ingredients, and instructions.");
      return;
    }
```

```
    try {
      await onSubmit(formData);
      if (successMessage) setMessage(successMessage);
      if (resetOnSuccess) setFormData(EMPTY_RECIPE_FORM);
    } catch (err) { setFormError(err.message); }
  };

  // ... form JSX renders fields for name, category, description,
  //     prepTime, cookTime, servings, ingredients, instructions
}

export default RecipeForm;
```

---

`src/pages/`

`Home.jsx`

Landing page. Reads `recipes` from the `useRecipes` hook and displays the first item as a featured recipe preview. Contains two CTA links: Add a Recipe and Browse Recipes.

```
import { Link } from "react-router-dom";
import { useRecipes } from "../hooks/useRecipes";

function Home() {
  const { recipes, isLoading, error } = useRecipes();
  const featuredRecipe = recipes[0];

  return (
    <section className="stack-gap">
      <article className="panel hero-panel">
        <p className="eyebrow">Plan your next meal</p>
        <h2>Build your personal recipe library.</h2>
        <p>Add your own dishes, keep one-click details, and browse everything
 in one place.</p>
        <div className="button-row">
          <Link className="button" to="/add">Add a Recipe</Link>
          <Link className="button button-ghost" to="/recipes">Browse Recipes<
/Link>
        </div>
      </article>
      <article class="panel">
        <h3>Featured Recipe</h3>
        {isLoading && <p>Loading recipes...</p>}
        {error && <p class="message error">{error}</p>}
        {!isLoading && !error && featuredRecipe && (
          <div class="featured-card">
            <p class="recipe-category">{featuredRecipe.category}</p>
            <h4>{featuredRecipe.name}</h4>
```

```
            <p>{featuredRecipe.description}</p>
            <Link className="button button-ghost" to={`/recipes/${featuredRec
ipe.id}`}>
              Open Recipe Detail
            </Link>
          </div>
        )}
      </article>
    </section>
  );
}

export default Home;
```

---

AddRecipe.jsx

Renders `RecipeForm` and connects it to the `addRecipe` action from `useRecipes`. On success, navigates to the newly created recipe's detail page using `useNavigate`.

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import RecipeForm from "../components/RecipeForm";
import { useRecipes } from "../hooks/useRecipes";

function AddRecipe() {
  const navigate = useNavigate();
  const { addRecipe } = useRecipes();
  const [isSubmitting, setIsSubmitting] = useState(false);

  const handleAddRecipe = async (formData) => {
    setIsSubmitting(true);
    try {
      const newRecipe = await addRecipe(formData);
      navigate(`/recipes/${newRecipe.id}`);
    } finally { setIsSubmitting(false); }
  };

  return (
    <section className="panel stack-gap">
      <div>
        <p className="eyebrow">Create recipe</p>
        <h2>Add your own recipe</h2>
        <p>Save the ingredients and steps so you can revisit them anytime.</p>
      </div>
      <RecipeForm onSubmit={handleAddRecipe} isSubmitting={isSubmitting} />
    </section>
  );
}
```

```
export default AddRecipe;
```

---

### BrowseRecipes.jsx

Displays all recipes in a responsive card grid with live client-side filtering. Two filter controls are provided: - A text `<input type="search">` that matches against recipe name, ingredients, and category - A `<select>` dropdown that restricts results by category

Filtering uses `useMemo` so it only recalculates when the recipe list, query, or category changes.

```
import { useMemo, useState } from "react";
import RecipeCard from "../components/RecipeCard";
import { CATEGORIES } from "../data/recipeSeed";
import { useRecipes } from "../hooks/useRecipes";

function matchesSearch(recipe, searchTerm) {
  const text = [recipe.name, recipe.category, recipe.ingredients.join(" ")].j
oin(" ").toLowerCase();
  return text.includes(searchTerm);
}

function BrowseRecipes() {
  const { recipes, isLoading, error } = useRecipes();
  const [query, setQuery] = useState("");
  const [category, setCategory] = useState("All");
  const q = query.trim().toLowerCase();

  const filteredRecipes = useMemo(() =>
    recipes.filter((r) =>
      (category === "All" || r.category === category) && (!q || matchesSearch
(r, q))
    ), [recipes, category, q]);

  return (
    <section className="stack-gap">
      <article className="panel">
        <p className="eyebrow">Browse and search</p>
        <h2>Find a recipe fast</h2>
        <div className="filter-bar">
          <input type="search" value={query}
            onChange={(e) => setQuery(e.target.value)}
            placeholder="Search by name, ingredient, or category" />
          <select value={category} onChange={(e) => setCategory(e.target.valu
e)}>
            <option value="All">All Categories</option>
            {CATEGORIES.map((c) => <option key={c} value={c}>{c}</option>)}
```

```
          </select>
        </div>
      </article>
      {isLoading && <p>Loading recipes...</p>}
      {error && <p className="message error">{error}</p>}
      {!isLoading && !error && filteredRecipes.length > 0 && (
        <div className="recipe-grid">
          {filteredRecipes.map((r) => <RecipeCard key={r.id} recipe={r} />)}
        </div>
      )}
      {!isLoading && !error && filteredRecipes.length === 0 && (
        <article className="panel">
          <h3>No recipes found</h3>
          <p>Try another keyword or category to find what you are looking for.
</p>
        </article>
      )}
    </section>
  );
}

export default BrowseRecipes;
```

---

`RecipeDetail.jsx`

The most complex page. Loads a recipe by the `:id` route param — first checking the in-memory context state (fast path), then falling back to a direct `getRecipeById` API call. Handles three modes: - **View mode** — shows full recipe with Edit and Delete action buttons - **Edit mode** — replaces the view with a `RecipeForm` pre-populated via `initialData`; a Cancel button returns to view mode - **Not found** — renders a fallback message with a link back to browse

Delete triggers a `window.confirm` prompt, calls `removeRecipe`, and navigates to `/recipes` on success.

The `useEffect` cleanup pattern (`isMounted` flag) prevents state updates on unmounted components during navigation.

```
import { useEffect, useState } from "react";
import { Link, useNavigate, useParams } from "react-router-dom";
import RecipeForm from "../components/RecipeForm";
import { useRecipes } from "../hooks/useRecipes";
import { getRecipeById } from "../services/recipesApi";

function RecipeDetail() {
  const navigate = useNavigate();
  const { id } = useParams();
  const { recipes, isLoading: isRecipesLoading, error: recipesError,
```

```
    editRecipe, removeRecipe } = useRecipes();

  const [recipe, setRecipe] = useState(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState("");
  const [isEditing, setIsEditing] = useState(false);
  const [isSaving, setIsSaving] = useState(false);
  const [isDeleting, setIsDeleting] = useState(false);

  useEffect(() => {
    let isMounted = true;
    async function loadRecipe() {
      setIsLoading(true); setError("");
      const local = recipes.find((r) => String(r.id) === String(id));
      if (local) {
        if (isMounted) { setRecipe(local); setIsEditing(false); setIsLoading
(false); }
        return;
      }
      try {
        const fetched = await getRecipeById(id);
        if (isMounted) { setRecipe(fetched); setIsEditing(false); }
      } catch (e) { if (isMounted) setError(e.message); }
      finally { if (isMounted) setIsLoading(false); }
    }
    loadRecipe();
    return () => { isMounted = false; };
  }, [id, recipes]);

  // Handlers for update and delete, edit/view mode toggle,
  // not-found and loading states, and full recipe detail JSX
}

export default RecipeDetail;
```